FIXLY — System Architecture & Specification

Autonomous Incident Detection & Self-Healing Platform | Hackathon Proposal (Node.js & Tailwind CSS)

---

1. Executive Summary & Vision

Fixly is an enterprise-grade, AI-driven autonomous incident detection and self-healing platform. Designed for modern cloud applications and remote server infrastructures, Fixly continuously monitors target server environments over secure SSH, ingests live application logs and system vitals (CPU, RAM, Disk), deduplicates repeated errors in real time, diagnoses root causes using Advanced Generative AI, and automatically generates, tests, and commits verified code fixes directly to Git repositories.

Built on a high-performance JavaScript stack (Node.js backend, React + Vite + Tailwind CSS frontend), Fixly combines real-time streaming WebSockets, intelligent SHA-256 error fingerprinting, automated Git branching, and a role-aware dashboard to reduce Mean Time to Detection (MTTD) and Mean Time to Resolution (MTTR) from hours down to seconds.

2. Technology Stack & Specifications

System Layer	Technology Selection	Technical Rationale
Language & Runtime	JavaScript (Node.js v20+)	High asynchronous I/O performance, unified JavaScript stack across client & server.
Frontend Dashboard	React 18+ with Vite & Tailwind CSS	Utility-first responsive design, lightning-fast HMR, modular component architecture.
Backend Services API	Express / Fastify (Node.js)	Lightweight HTTP routing, native JSON parsing, robust middleware architecture.
Real-Time Engine	WebSockets (ws / socket.io)	Bi-directional low-latency event broadcasting for live incident feeds and vitals gauges.

Database & ODM Layer	MongoDB with Mongoose ORM	Schema validation models, index acceleration, and atomic document updates.
SSH Transport	node-ssh / ssh2	Encrypted key-based remote log streaming and vitals command execution.
Version Control Integration	simple-git	Programmatic Git repository management, automated feature branch creation, and PR pushing.

3. System Modules & Functional Architecture

Fixly is architected into 4 decoupled, parallel modules in JavaScript. Each module maintains strict domain boundaries to enable multi-agent parallel development.

Module 1: Server Connection & Monitoring (User 1 Scope)

- **Key-Based SSH Transport (ssh_client.js):** Connects to remote target server via node-ssh private key authentication.
- **Continuous Log Streaming (log_reader.js):** Tails application logs line-by-line in real time, filtering error stack traces.
- **Server Vitals Reader (vitals_reader.js):** Periodically captures CPU %, RAM %, and Disk % metrics.
- **SHA-256 Deduplication Engine (dedup_engine.js):** Normalizes logs by stripping variable IDs/timestamps and generates a SHA-256 fingerprint hash.
- **Real-Time Broadcaster (ws_broadcaster.js):** Emits live incident:created, incident:updated, and vitals:updated events over WebSocket.

Module 2: AI Logic & Version Control Integration (User 2 Scope)

- **AI Diagnosis Engine (diagnosis.js):** Analyzes error context and stack traces to return severity, root cause, confidence score, and fixability.
- **AI Code Patch Proposer (code_fixer.js):** Prompts AI for precise code fixes and outputs line-by-line unified diff patches.
- **Git Manager & PR Creator (git_client.js):** Creates dedicated branch fix/inc-<id> via simple-git, commits patch, and pushes Pull Request.
- **Recovery Verifier (recovery.js):** Monitors log stream post-fix. Auto-resolves incident when zero recurrences are verified.

Module 3: Interface & User Experience (User 3 Scope)

- **Live Incident Feed (LiveFeed.jsx):** Real-time updating visual feed styled with Tailwind CSS severity badges.
- **Server Vitals Dashboard (ServerVitalsWidget.jsx):** Interactive progress bars for real-time CPU, RAM, and Disk metrics.
- **Tracked Issues Kanban (IssuesBoard.jsx):** Categorized board managing OPEN, IN_PROGRESS, and RESOLVED states.
- **Code Diff Inspector (DiffViewer.jsx):** Line-by-line visual before/after code comparison modal.

Module 4: Auth, Access Control & System Lead (User 4 Scope)

- **JWT Auth & RBAC (auth_service.js):** Issues authenticated JWT tokens carrying ADMIN, OPERATOR, and READ_ONLY role claims.
- **Backend Settings API (settings_service.js):** Provides encrypted GET/PUT /api/settings endpoints for masked token management.
- **Shared Mongoose Models (src/models/):** Maintains MongoDB Mongoose models for Users, Incidents, Occurrences, Vitals, Settings, and Audit Logs.
- **Demo Target Environment (target_environment/):** Isolated target server app containing reproducible error scenarios.

4. End-to-End Execution Workflow

- **Step 1 (Ingestion):** An unhandled exception occurs on the target application server. Module 1 reads the log line over SSH.
- **Step 2 (Deduplication):** Module 1 strips dynamic variables, calculates the SHA-256 fingerprint, and checks MongoDB Mongoose models.
- **Step 3 (AI Diagnosis & Git Patching):** Module 2 analyzes the stack trace, generates a root-cause diagnosis, creates a unified diff patch, commits to branch fix/inc-<id>, and opens a Git Pull Request.
- **Step 4 (Live Dashboard Stream):** Module 3 instantly updates the React + Tailwind CSS dashboard over WebSocket with the incident details.
- **Step 5 (Auto-Recovery Verification):** Module 2 monitors target server logs for 5 minutes post-fix. When zero recurrences are verified, the incident auto-resolves.

5. Non-Functional Specs & Hackathon Impact

- **Performance:** Incident detection to WebSocket dashboard streaming completes in < 500ms. AI diagnosis and Git patch creation complete in < 8 seconds.
- **Security & Encryption:** Key-based SSH authentication, AES-256 encrypted access tokens in MongoDB, JWT HTTP Bearer headers, and masked settings.
- **Competitive Innovation:** Combines real-time monitoring, LLM-based code patching, and version-controlled Git workflows into a single autonomous closed loop.